

chia-hackathon-proposal

Compute-Hour Efficient Coverage Growth: Agentic Sequencing of Verification Engines

Track: (relates to *autonomous construction of verification collateral*)

Contact: Kapil Bansal, Xorvis AI. (kapil@xorvis.ai)

1. Overview

Verification reaches coverage by carpet bombing. Its an overkill of using all arsenal at hand. Several approaches combined in test-space (CRV, directed, formal..), model-space(simulation, FGPA, various DUT hierarchies), evolving Checkers & test tuning based on learning, Pruning of unreachable scenarios, strategies for hard to hit scenarios.

A large constrained-random regression is launched, it is good in beginning but coverage plateaus after some coverage pass rate & increasingly compute inefficient further. The last few percent consumes most of the budget, for two reasons: random stimulus keeps re-covering ground it already holds, and a significant fraction of the remaining cover points are unreachable — the regression burns hours chasing targets that do not exist.

The engines themselves are not weak. The allocation is. A team has an arsenal — random, constrained-random, constraint tuning, directed tests, formal — with very different cost and yield profiles at various stages of verification, and the mix is typically adhoc & human wisdom based.

This proposal builds a CHIA loop that dynamically strives to keep coverage progress compute hour efficient. An agent acts as verification strategy planner: it observes the coverage state and the measured coverage-per-hour of each engine so far, then allocates the next block of compute to the action most likely to move the curve. **The deliverable is the coverage growth curve against cumulative simulation-CPU-hours, agent-planned versus conventional fixed-recipe regression.**

2. Methodology

DUT. MediumBOOM in a Chipyard SoC — every dynamic engine below already exists as a CHIA node, so effort goes into the planner and the formal integration.

Frozen coverage model. Verilator line, toggle, and fixed hand-written SVA cover points over the LSU, branch predictor, and issue queues. Frozen before any arm runs and collected on a worker the agent has no credentials for. The planner may request a filtered view of what is uncovered; it cannot alter the yardstick.

The arsenal. Five actions the planner can allocate to:

Action	Character	Cost	Yield profile
Torture	Carpet bombing — pure random	low	broad, saturates early
riscv-dv	Area fire — constrained-random regression	medium	strong mid-range
Constraint tuning	Re-aiming the artillery — adjust weights, test class, knobs	very low	reopens a stalled engine
Directed	Sniper — agent-authored assembly at named uncovered points	high per test	narrow, late-stage
Formal (SymbiYosys)	Prove or retire	medium	see below

The loop. Each round: (i) query uncovered points, per-action yield history, and the current reachable denominator; (ii) agent emits an allocation — action, target, parameters, hour budget; (iii) CHIA fans the batch across the cluster; (iv) coverage merged, unreachable points retired, yield table updated; (v) repeat.

Baseline. riscv-dv at default weights scaled out until plateau, then a fixed directed suite. This is what teams actually do, and it is what we must beat on the curve.

Accounting. Simulation-CPU-hours from Ray task timings via `chia viz-profile`; formal solver hours counted in the same currency. LLM wall-clock and tokens recorded and reported separately, never charged against the compute budget. Three seeds per arm.

(optional extended goal) New plumbing, scoped to week 1. CHIA has no formal node. We contribute

`SymbiYosysNode` (Yosys/SBY over Chipyard-generated Verilog, BMC and induction modes)

— useful to the community independent of this result.

Why this needs CHIA. Five actions with different runtimes, container images, and resource demands, switched mid-run by a planner, is heterogeneous scheduling — not a shell script or simple DAG.

3. Expected Results

- 1. **The curve** — coverage versus cumulative compute-hours for both arms, with seed variance. Expected shape: comparable early, diverging sharply in the last mile.
- 2. **Hours to 90 / 95 / 99%** of baseline-final coverage, as the headline ratio.
- 3. **The learned policy** — which action the planner favoured at which coverage level, and how much of the late-stage saving came from formal retirement versus directed targeting. This is the transferable insight even if the ratio is modest.
- 4. **Reusable CHIA blocks:** `SymbiYosysNode` , `EngineYieldNode` (coverage-per-hour tracking), `CoverageMeterNode` (isolated frozen collection), submitted upstream.
- 5. Paves path to more autonomous verification.

Primary risk: the planner does no better than a fixed schedule. Mitigation and consolation — the per-action yield curves are a measurement nobody has published, and a week-1 pilot at small budget shows early whether the profiles differ enough to exploit.

4. Cost Estimate

Item
Simulation compute
Formal solver compute
Head node + orchestration
LLM API (planner + directed authoring + property drafting)
S3 storage and egress

Around \$1000